

Input and output

A hands-on course in building agents by building their harness

Lesson 2

The model takes tokens in and gives tokens out

It can't read what you type, and it can't run a command.

Input and output are how the harness connects it to everything else.

Input

Input is how inputs are gathered: from a person, like a message, a question or a correction, or from the world, like what happened when a command ran.

Output

Output is how the model's outputs are handled: shown to a person, or run as a tool.

Afferent and efferent

Input is the harness's afferent pathway, carrying signals in.

Output is its efferent pathway, carrying actions out.

Two primitives, taught together

They're built independently, but they're two ends of one exchange, so they're taught together.

The exchange runs through tools.

The model can't act, but it can ask

The model can't act, but it can ask: its response can include a request to use a tool, a name and arguments in a shape the harness described to it.

The model asks; the harness acts

Output runs the request, and what happened comes back as input.

The model asks; the harness acts.

Tools aren't strictly required

You don't strictly need tools to have an agent.

A model that can only talk still takes things in and responds.

But without tools, the only thing it can change is what a person reads.

Where they sit

Here's where they sit around Lesson 1's call:

```
person or world → input → request → model interface → response → output → person or world
```

Lesson 1 had two stubs

In Lesson 1, both ends were stubs: a hardcoded question going in, and a raw dump of the response coming out.

This lesson makes each end real on its own, then puts them together.

Input: `input.py`

Here's Lesson 1's harness with real input.

The output is still Lesson 1's stub.

The model interface section is Lesson 1's, unchanged.

Two sections are new.

The new input section

```
# — input —————  
def read(prompt):                                # input: from a person  
    while True:  
        print(prompt, end="", flush=True)  
        line = sys.stdin.readline()  
        if not line: return "/q"  
        if line.strip(): return line.rstrip("\n")  
        prompt = "> "  
input = " ".join(sys.argv[1:]) or read("> ")  
if input == "/q": sys.exit()
```

end of input (Ctrl-D): nothing more is coming
Enter on an empty line: a fresh prompt, as in a terminal

`input` is whatever comes in

```
input = " ".join(sys.argv[1:]) or read("> ")
```

— `input` — gathers what goes to the model.

`input` is whatever comes in: here, the words on the command line, or a line typed at the `>` prompt.

It isn't assumed to be a task

It isn't assumed to be a task or a question; it's just input, and it goes into the request as the `user` message.

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": input}])
```

read()

```
def read(prompt):                                     # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
```

`read()` is the afferent pathway from a person: it prints a prompt and reads a line.

An empty line sends nothing

```
if not line: return "/q"           # end of input (Ctrl-D): nothing more is coming
if line.strip(): return line.rstrip("\n") # Enter on an empty line: a fresh prompt, as in a terminal
prompt = "> "
```

Press Enter on an empty line and it gives you a fresh `>` on the next line, as a terminal does, so you can make space as often as you like; nothing is sent.

Ctrl-D returns `/q`

```
if not line: return "/q"                                # end of input (Ctrl-D): nothing more is coming
```

```
if input == "/q": sys.exit()
```

When the input ends (Ctrl-D) it returns `/q` , so the harness can stop.

The one tool

```
tools = [{"name": "bash", "description": "Run shell command – the whole system is in reach", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}]]
```

— output: the one tool — describes one tool, `bash`, and goes in the request so the model knows it can ask for it.

Nothing here can run it yet.

Output is still the stub

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": input}])  
print(output.model_dump_json(indent=2))
```

Running it is output, which is still the stub: `output` is the whole response, dumped.

Run `input.py`

From the root of the repo, give it input that needs a tool:

```
uv run lessons/02-input-and-output/input.py "how many lines are in README.md?"
```

One run: the model's request

Here's one run.

(Shortened: only the `content` field is shown.)

```
"content": [  
  {  
    "id": "toolu_01MZ8CLRS3qnG1TjJz5AoCX1",  
    "caller": {  
      "type": "direct"  
    },  
    "input": {  
      "cmd": "wc -l README.md"  
    },  
    "name": "bash",  
    "type": "tool_use",  
    "toolset_name": null  
  }  
],
```

One run: `stop_reason`

(Shortened: only the `stop_reason` line is shown.)

```
"stop_reason": "tool_use",
```

The input went in, and the model answered it the only way it can: with a `tool_use` block asking to run `wc -l README.md`, and a `stop_reason` of `tool_use`, which means it stopped to wait for the result.

The model asked; nothing acted

The output stub dumps that request to the screen, and nothing runs it.

The model asked; nothing acted.

Output: `output.py`

Here's Lesson 1's harness with real output.

The input is still Lesson 1's hardcoded question.

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": "What's in this directory?"}]).content
```

`tools` is what output can do

```
tools = [{"name": "bash", "description": "Run shell command – the whole system is in reach", "input_schema": {"type": "object", "properties": {"cmd": {"type": "string"}}, "required": ["cmd"]}}]
```

Each tool has a name, a description the model reads to decide when to use it, and a schema for its arguments.

The tools travel in the request

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": "What's in this directory?"}]).content
```

The descriptions travel in the request, `tools=tools`, so the model knows what it can ask for.

Warning: `bash` is everything

With `bash`, anything you can do from a command line, the model can ask for, which is why the setup warns you to run it somewhere you can afford to lose.

output is a list of blocks

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": "What's in this directory?"}]).content
```

output is now the content of the response, `call(...).content` : a list of blocks.

for block in output

```
for block in output:                                     # output: show text, run tool requests
```

for block in output handles each one where it belongs.

A `text` block is shown

```
if block.type == "text":  
    print(block.text)
```

A `text` block is printed for the person.

A `tool_use` block is a request

```
if block.type == "tool_use":  
    print(f"$ {block.input['cmd']}")
```

A `tool_use` block is a request to run something.

Thinking blocks go nowhere

Thinking blocks go nowhere.

Why `max_tokens` is 16384

`max_tokens` is 16384 here and from now on, to leave room for the model to think before it asks: a response cut off in the middle of a tool request would leave `block.input` without a `cmd`, and this code would crash.

`input_output.py`, further down, checks for that instead.

`subprocess.run(...)` runs the command

```
done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
```

`subprocess.run(...)` runs the command.

`stderr=subprocess.STDOUT` merges errors into the output, in the order they happened.

The person sees what ran

```
print(f"$ {block.input['cmd']}")
```

```
print(done.stdout)
```

The command and what it printed go to the person, so they can see what ran.

Run `output.py`

From the root of the repo:

```
uv run lessons/02-input-and-output/output.py
```

One run: `ls -la`

Here's one run.

(Listing shortened to its first lines.)

```
$ ls -la
total 156
drwxr-xr-x 9 root root 4096 Oct  6 16:29 .
drwxr-xr-x 5 root root 4096 Oct  5 14:47 ..
-rw-r--r-- 1 root root  104 Oct  5 22:09 .env.example
drwxr-xr-x 8 root root 4096 Oct  6 16:29 .git
-rw-r--r-- 1 root root  113 Oct  6 03:58 .gitignore
```

The model asked for `ls -la`, and output ran it.

The listing went to the screen, and nowhere else.

Together: `quark.py`

Here's `input.py` and `output.py` in one file: input's `read()` and `input` going in, output's `tool` and `handling` coming out.

Lesson 1's `dump` is gone, because output now handles the response.

Input's half of `quark.py`

```
# — input —————
def read(prompt):                                # input: from a person
    while True:
        print(prompt, end="", flush=True)
        line = sys.stdin.readline()
        if not line: return "/q"                # end of input (Ctrl-D): nothing more is coming
        if line.strip(): return line.rstrip("\n") # Enter on an empty line: a fresh prompt, as in a terminal
        prompt = "> "
input = " ".join(sys.argv[1:]) or read("> ")
if input == "/q": sys.exit()
```

Output's half of `quark.py`

```
output = call(max_tokens=16384, tools=tools, messages=[{"role": "user", "content": input}]).content

input = []
for block in output:                                # output: show text, run tool requests
    if block.type == "text":
        print(block.text)
    if block.type == "tool_use":
        print(f"$ {block.input['cmd']}")
        done = subprocess.run(block.input["cmd"], shell=True, stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True)
        print(done.stdout)
        input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.returncode})"}) # input: from the world
```

One new piece

And there's one piece neither file could have on its own:

```
input = []
```

`input = []` is input again, this time from the world.

What the tool printed

Once output runs a tool, what the tool printed is something the model should know about.

```
input.append({"type": "tool_result", "tool_use_id": block.id, "content": done.stdout or f"(exit {done.returncode})"}) # input: from the world
```

The `tool_result`

Each `tool_result` puts it in the shape the model reads, naming the `tool_use_id` it answers, and a command that prints nothing still reports its exit code.

Same name, on purpose

It's the same name as the input from a person, on purpose: input is whatever comes in, from a person or from the world.

That's the exchange: the model asks, output acts, input brings back what happened.

Run `quark.py`

From the root of the repo:

```
uv run lessons/02-input-and-output/quark.py "how many lines are in README.md?"
```

Here's one run:

```
$ wc -l README.md  
333 README.md
```

Same input, same request

It's the same input as `input.py`, and the model asked for the same `wc -l README.md`.

This time output ran it.

The model didn't say anything before asking; sometimes it does.

Run it with no input on the command line and it prompts you with `>` instead.

The model never sees the 333

Notice what never happens: the model never sees the 333.

It's sitting in `input`, and nothing sends it.

What else input and output can be

quark gathers input from a terminal and runs one tool on the same machine, but these primitives hold more in other harnesses.

Input can differ

- **Where a person's input comes from.** Command-line arguments, a prompt, a pipe, a chat app, a webhook, a schedule.
- **What it can be.** Text, images, files.
- **What happens to bad input.** A blank message can be asked for again instead of sent.
- **What comes back from the world.** What a tool printed, its exit code, and whether it failed.

Output can differ: where and how

- **Where it goes.** A terminal, a file, a chat message, a pull request.
- **How it reaches a person.** All at once, or as it's produced.
- **Which tools exist.** One general tool, or many specific ones, each request routed by name.
- **Where tools run.** This machine, a container, a remote machine, or a tool the provider hosts.

Output can differ: when it runs

- **How tool requests run.** One after another, or all at the same time.
- **When not to run.** A tool request the model was cut off in the middle of is incomplete, and running half a command is worse than running none.
- **What happens when a tool fails or hangs.** Stop it after a timeout, and report the failure instead of crashing.

MCP

Some of this is a product on its own.

MCP servers package tools behind one protocol, so a harness can run tools someone else built without writing them.

That part of MCP is output you plug in.

A fuller file: `input_output.py`

Here's input and output that do more of that, in `input_output.py`.

The input is a Telegram chat, and the output goes back to that chat and to a tool executor that runs every tool request at once.

Where a person's input comes from

`receive()` asks Telegram for new messages and waits until one arrives, then hands back its text and which chat it came from.

It takes one message and stops; answering message after message would be control flow.

Where output goes

`send()` posts the model's words, and each tool's result, back to the chat the message came from.

Which tools exist

Two, `bash` and `read_file`.

`executors` maps each tool's name to the code that runs it.

An unknown tool

A request for a tool that isn't there gets an error back instead of crashing the harness.

How tool requests run

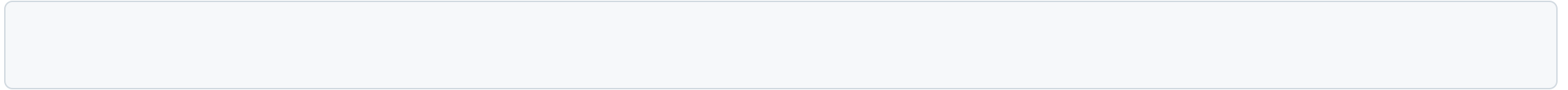
`asyncio.gather` starts every request in the response at once and waits for all of them.

Three commands that each take two seconds finish in two seconds, not six.

When not to run

If the response hit `max_tokens`, its last tool request may be incomplete, so it isn't run.

When a tool hangs



`bash` is stopped after 30 seconds.

When a tool fails

Failures come back as results with `is_error` set, and a non-zero exit code is added to the output.

Run `input_output.py`

To run it, make a bot by messaging [@BotFather](#) on Telegram, put its token in `.env` as `TELEGRAM_BOT_TOKEN=...`, then:

```
uv run lessons/02-input-and-output/input_output.py
```

Send your bot a message.

One run: three tools at once

Here's one run.

For this run, Telegram was swapped for a stand-in that prints each message, labeled by which way it went; the model and the tools were real:

```
[telegram → bot] check the python3 version, the git version and the first line of pyproject.toml, all three at once
```

```
[bot → telegram] → bash {"cmd": "python3 --version"}
```

```
Python 3.13.14
```

```
[bot → telegram] → bash {"cmd": "git --version"}
```

```
git version 2.43.0
```

```
[bot → telegram] → bash {"cmd": "head -n 1 pyproject.toml"}
```

```
[project]
```

Three requests, run together

One message asked for three things, the model asked for three tools in one response, and the executor ran them together.

The results went to the chat.

As with `quark.py`, the model never sees them.

The rule

The rule: input gathers what goes to the model, from a person or from the world.

Output handles what comes back, shown to a person or run as a tool.

A tool is the model asking and the harness acting.

What they never do

Notice what input and output never do.

Getting the request to the model and the response back is the model interface.

When to call, and whether a result goes back around, is control flow.

How what input gathers is presented in the request is context.

What each one only does

Input only gathers what goes in, from a person or the world, and output only handles the response, showing it to a person or running a tool.

What's missing

What's missing: the model asked for `wc -l`, it ran, and the model never saw what it found.

The path ends at output.

```
person or world → input → request → model interface → response → output → person or world
```

Something has to send the result back to the start and decide to go again.

That's control flow.

Next: Control flow

Lesson 3